

Lab02_Sarah_Mariuci

Sarah Mariuci

Lab 02 ME315

```
library(readr)
library(dplyr)
```

Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

filter, lag

The following objects are masked from 'package:base':

intersect, setdiff, setequal, union

```
library(ggplot2)
```

Tarefa

- Lendo 100.000 observações por vez (se você acredita que seu computador não tem memória suficiente, utilize 10.000 observações por vez), **determine o percentual de vôos por Cia. Aérea que apresentou atraso na chegada (ARRIVAL_DELAY) superior a 10 minutos.** As companhias a serem utilizadas são: **AA, DL, UA e US.** A estatística de interesse deve ser calculada para *cada um dos dias de 2015*. Para a determinação deste percentual de atrasos, apenas verbos do pacote dplyr e comandos de importação do pacote readr podem ser utilizados. Os resultados para cada Cia. Aérea devem ser apresentados em um formato de **calendário**.

Instruções

1. Quais são as estatísticas suficientes para a determinação do percentual de vôos atrasados na chegada ($ARRIVAL_DELAY > 10$)?

Resposta: Para calcular esse percentual de vôos atrasados precisamos contar:

- Quantos vôos tiveram $ARRIVAL_DELAY > 10$
- Quantos vôos tiveram no total

em cada um dos dias/mês/companhia.

2. Crie uma função chamada `getStats` que, para um conjunto de qualquer tamanho de dados provenientes de `flights.csv.zip`, execute as seguintes tarefas (usando apenas verbos do `dplyr`):
 - a. Filtre o conjunto de dados de forma que contenha apenas observações das seguintes Cias. Aéreas: AA, DL, UA e US;
 - b. Remova observações que tenham valores faltantes em campos de interesse;
 - c. Agrupe o conjunto de dados resultante de acordo com: dia, mês e cia. aérea;
 - d. Para cada grupo em b., determine as estatísticas suficientes apontadas no item 1. e os retorne como um objeto da classe `tibble`;
 - e. A função deve receber apenas dois argumentos: i. `input`: o conjunto de dados (referente ao lote em questão); ii. `pos`: argumento de posicionamento de ponteiro dentro da base de dados. Apesar de existir na função, este argumento não será empregado internamente. **É importante** observar que, sem a definição deste argumento, será impossível fazer a leitura por partes.

```
library(readr)
library(dplyr)

getStats <- function(input, pos) {
  input |>
    filter(
      AIRLINE %in% c("AA", "DL", "UA", "US"),
      !is.na(ARRIVAL_DELAY)
    ) |>
    group_by(YEAR, MONTH, DAY, AIRLINE) |>
    summarise(
      voos_atrasados = sum(ARRIVAL_DELAY > 10),
      total_voos = n(),
      .groups = "drop"
    )
}
```

3. Utilize alguma função `readr::read_***_chunked` para importar o arquivo `flights.csv.zip`.
 - a. Configure o tamanho do lote (chunk) para 100 mil registros;
 - b. Configure a função de callback para instanciar DataFrames aplicando a função `getStats` criada acima;
 - c. Configure o argumento `col_types` de forma que ele leia, diretamente do arquivo, apenas as colunas de interesse (veja nota de aula para identificar como realizar esta tarefa);

```
library(readr)

df_stats <- read_csv_chunked(
  file = "flights.csv",
```

```

callback = DataFrameCallback$new(getStats),
chunk_size = 100000,
col_types = cols_only(
  YEAR = col_integer(),
  MONTH = col_integer(),
  DAY = col_integer(),
  AIRLINE = col_character(),
  ARRIVAL_DELAY = col_double()
)
)

head(df_stats)

```

```

# A tibble: 6 × 6
  YEAR MONTH DAY AIRLINE voos_atrasados total_voos
  <int> <int> <int> <chr>          <int>      <int>
1  2015     1     1 AA             514      1379
2  2015     1     1 DL             132      1558
3  2015     1     1 UA             367      1269
4  2015     1     1 US             201      1028
5  2015     1     2 AA             649      1508
6  2015     1     2 DL             406      2261

```

4. Crie uma função chamada `computeStats` que:
 - a. Combine as estatísticas suficientes para compor a métrica final de interesse (percentual de atraso por dia/mês/cia aérea);
 - b. Retorne as informações em um tibble contendo apenas as seguintes colunas:
 - i. Cia: sigla da companhia aérea;
 - ii. Data: data, no formato AAAA-MM-DD (dica: utilize o comando `as.Date`);
 - iii. Perc: percentual de atraso para aquela cia. aérea e data, apresentado como um número real no intervalo $[0,1]$.

```

computeStats <- function(stats) {
  stats |>

  group_by(YEAR, MONTH, DAY, AIRLINE) |>

  summarise(
    atrasados_total = sum(voos_atrasados),
    voos_total = sum(total_voos),
    .groups = "drop"
  ) |>

  mutate(

```

```

    Cia = AIRLINE,
    Data = as.Date(paste(YEAR, MONTH, DAY, sep = "-")),
    Perc = atrasados_total / voos_total
  ) |>

  select(Cia, Data, Perc)
}

```

5. Produza um mapa de calor em formato de calendário para cada Cia. Aérea.
 - a. Instale e carregue os pacotes ggcal e ggplot2.
 - b. Defina uma paleta de cores em modo gradiente. Utilize o comando `scale_fill_gradient`. A cor inicial da paleta deve ser #4575b4 e a cor final, #d73027. A paleta deve ser armazenada no objeto `pal`.
 - c. Crie uma função chamada `baseCalendario` que recebe 2 argumentos a seguir: `stats` (tibble com resultados calculados na questão 4) e `cia` (sigla da Cia. Aérea de interesse). A função deverá:
 - d. Criar um subconjunto de `stats` de forma a conter informações de atraso e data apenas da Cia. Aérea dada por `cia`.
 - i. Para o subconjunto acima, montar a base do calendário, utilizando `ggcal(x, y)`. Nesta notação, `x` representa as datas de interesse e `y`, os percentuais de atraso para as datas descritas em `x`.
 - ii. Retornar para o usuário a base do calendário criada acima.
 - iii. Executar a função `baseCalendario` para cada uma das Cias. Aéreas e armazenar os resultados, respectivamente, nas variáveis: `cAA`, `cDL`, `cUA` e `cUS`.
 - e. Para cada uma das Cias. Aéreas, apresente o mapa de calor respectivo utilizando a combinação de camadas do ggplot2. Lembre-se de adicionar um título utilizando o comando `ggtitle`. Por exemplo, `cXX + pal + ggtitle("Titulo")`.

```

library(ggcal)
baseCalendario <- function(stats, cia) {
  sub_stats <- stats |>
    filter(Cia == cia)

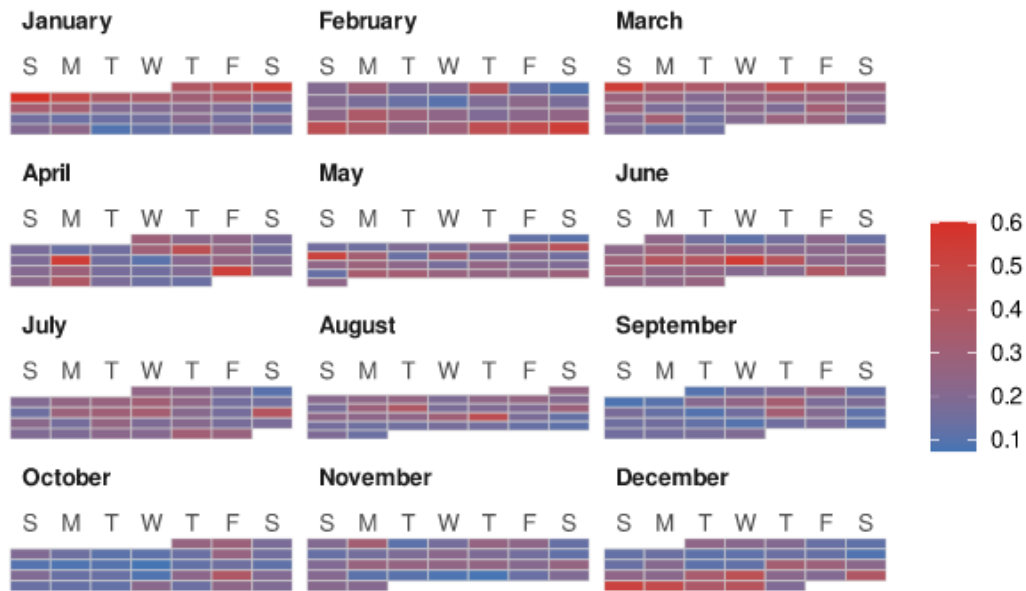
  calendario_base <- ggcal(sub_stats$Data, sub_stats$Perc)
  return(calendario_base)
}

cAA <- baseCalendario(computeStats(df_stats), "AA")
cDL <- baseCalendario(computeStats(df_stats), "DL")
cUA <- baseCalendario(computeStats(df_stats), "UA")
cUS <- baseCalendario(computeStats(df_stats), "US")

```

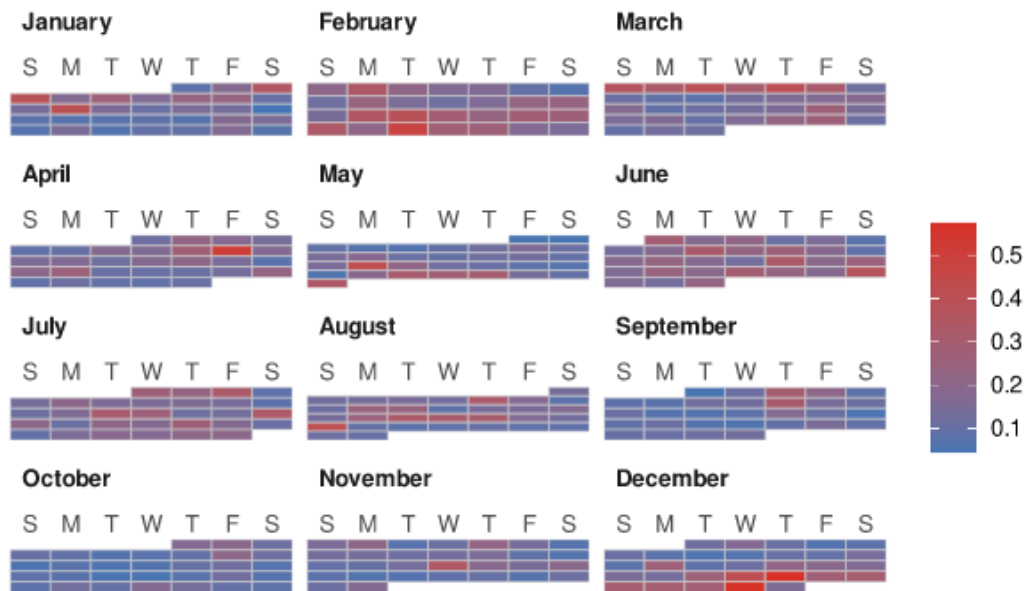
```
pal <- scale_fill_gradient(low = "#4575b4", high = "#d73027")
cAA + pal + ggtitle("Calendário de Atrasos 2015 - American Airlines (AA)")
```

Calendário de Atrasos 2015 - American Airlines (AA)



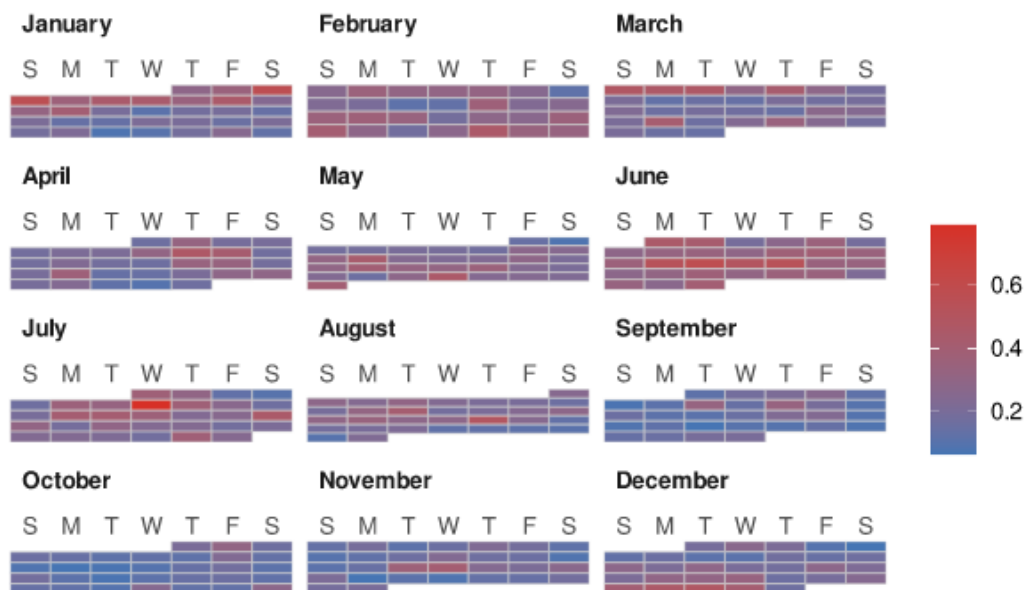
```
cDL + pal + ggtitle("Calendário de Atrasos 2015 - Delta Air Lines (DL)")
```

Calendário de Atrasos 2015 - Delta Air Lines (DL)



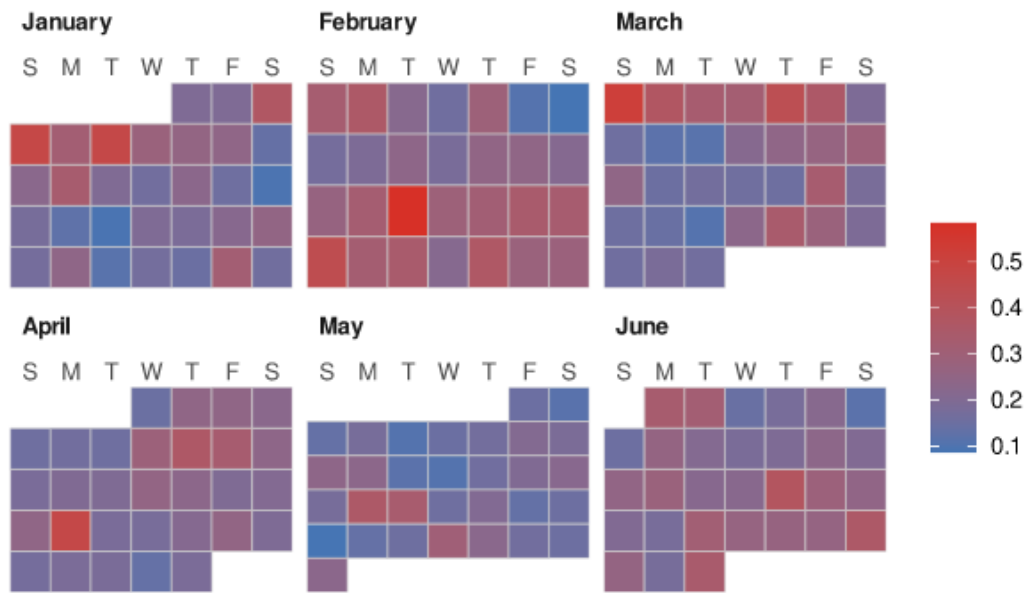
```
cUA + pal + ggtitle("Calendário de Atrasos 2015 - United Airlines (UA)")
```

Calendário de Atrasos 2015 - United Airlines (UA)



```
cUS + pal + ggtitle("Calendário de Atrasos 2015 - US Airways (US)")
```

Calendário de Atrasos 2015 - US Airways (US)



Horário de conclusão

```
Sys.setenv(TZ = "America/Sao_Paulo")  
Sys.time()
```

```
[1] "2026-08-24 22:29:07 -03"
```